

Dijkstra Sequence

Date: 2026-05-05



Chapter 1: 引言

1.1、问题描述

给定一个无向带权连通图 $G(V, E)$ ，以及 K 个由图的顶点组成的序列。需要判断每个给定的序列是否是一个合法的 Dijkstra 序列。

1.2、算法背景

- **Dijkstra 算法**：解决带权图中单源最短路径问题的贪心算法。它维护一个顶点路程数组，初始时起点距离为 0，其余为无穷大。算法循环“**寻找未确定顶点中距离最小者 -> 确定该点 -> 更新其邻居的最短路程**”的过程，直到所有顶点均被确定。
- **多个合法Dijkstra序列**：算法循环中，每次从未确定路径的点的集合中选取当前距离源点最短的顶点时，由于可能存在多个距离相同的最小顶点，可能产生的不同但均合法的序列
- **最小堆（优先队列，MinPQ）**：用数组实现的完全二叉树数据结构，该数据结构仅保证父节点取值小于等于两个子节点，用于在 $O(\log V)$ 时间内快速获取当前距离最小的顶点，在该project中用于稀疏图的最短路径优化

1.3、任务目标

- **建图**：根据输入的节点信息，通过邻接表构建无向连通图
- **给定序列的动态验证**：不主动生成最短路径，而是按照输入序列给定的顶点顺序，验证该点在当前状态下是否具备“距离最小”的资格
- **时间复杂度优化**：针对图的密度，实现 $O(|V|^2 + |E|)$ 的基础验证法与 $O(|E| \log |V|)$ 的优先队列验证法，并根据图的规模动态切换

1.4、问题分析

- **序列不唯一性**：由于可能存在多个未确定（Unknown）且距离皆为最小的顶点，同一张图的 **Dijkstra** 序列可能不止一个。因此不能简单地跑一遍算法然后全量对比，而是必须采用“**逐点验证是否合理**”的验证策略。
- **最小堆结构优化**：由于最小堆只能确保父节点**大于或等于**子节点，而 **Dijkstra** 序列可能出现因为两个顶点**距离一样且最小**产生多条路径，因此在使用最小堆优化时，传统的弹出堆顶

操作无法处理序列指定顶点与堆顶顶点距离相同但编号不同的情况。所以，我将判断序列中该顶点是否合法的依据改成了距离是否和堆顶距离相同，并清除已经被标记为确定的顶点。

- **最小堆结构修改**：传统的 Dijkstra 的最小堆优化方案中，当一个顶点的最短路径被更新后直接将一个全新的节点投入最小堆中，因此该优化方案最坏情况同一个顶点会将多个距离节点存入最小堆，会占用大量空间。为此，我引入了 `pos` 映射数组，用于当一个节点的最短距离被确定后可以直接**找到并改变**其在最小堆中的值并更新其在最小堆中的位置，以此来保证堆的大小 $\leq V$ ，虽然代码略显臃肿，但是节约了空间成本
-

Chapter 2：算法规范

2.1、数据结构

2.1.1 图与无向邻接表（Graph & Undirected Adjacency List）

- **无向邻接表表示的图**：无向邻接表是一个链表数组，数组每一个**链表**：连接的节点代表**下标对应顶点**与其临接的邻居的编号以及这条边的权重。以此**储存图的连通关系**，由于该图是无向连通图，所以同一条边要**同时**记录在两个顶点对应的邻接链表里

```

typedef struct AdjVNode *PtrToAdjVNode;
struct AdjVNode {
    int AdjV;          /* 邻居顶点编号 */
    int weigh;         /* 边权重 */
    PtrToAdjVNode next;
};

typedef struct Vnode {
    PtrToAdjVNode FirstEdge; /*指向该顶点其中一个邻居的边节点*/
} AdjList[MAXVERTEX];

struct {
    int Nv;             /*该图的节点数*/
    int Ne;             /*该图的边数*/
    AdjList G;          /*邻接表数组*/
} Graph;

```

2.1.2 状态表 (State Stable)

- **记录节点状态的数组**：通过一个独立的动态数组 `Vnode` 来追踪节点状态，包括每个顶点在运行过程中的**当前最短距离** `dist` 和该节点是否**被确定的标志** `know`
- **更新邻居节点的最短路径**：对于当前选定的最小距离顶点 V 和它的邻居 W ，如果 $\text{dist}[V] + \text{weight}(V, W) < \text{dist}[W]$ ，则更新 `dist[W]`

```

struct node {
    int know;          /* Status flag: 1 表示已确定最短路径, 0 表示未确定 */
    int dist;          /* 当前距离起点的位置 */
} Vnode[MAXVERTEX]; /*直接使用该结构体数组记录所有节点状态*/

```

3.1.3 带位置映射的最小堆 (Min PQ with Index Mapping)

- **最小堆结构**：构建最小堆的比较依据为该顶点的**最短距离**，但最小堆内每个节点**不仅存储距离**，还**存储顶点编号**。`pos` 位置映射数组起到**定位追踪**节点的作用，能够实现 $O(1)$ 的时间定位任意顶点在堆中的位置，用于更新顶点距离后直接更新该顶点在数组中的位置
- **最小堆优化原理**：在传统的Dijkstra思路中，当图极为稀疏时，边数远小于 $|V|^2$ ，则原始方

法的时间复杂度依然 $O(|V|^2)$,如果使用最小堆优化,每个顶点最多被提取一次 ($O(\log|V|)$), 每条边最多触发一次更新 ($O(|E|)$),最后时间复杂为 $O(|E|\log|V|)$,对于稀疏图远小于原始算法

```
struct MinPQNode {
    int v;          /* 顶点编号 */
    int dist;       /* 最短距离 (建堆的排序依据) */
};

struct {
    struct MinPQNode shortestPath[MAXVERTEX]; /* 堆数组, 从下标 1 开始 */
    int size;                                /* 当前堆内元素个数 */
    int pos[MAXVERTEX];                      /* 映射数组: 顶点 v 对应堆数组的下标 */
} MinPQ;
```

2.2、函数设计与实现

2.2.1 顶点状态初始化 (Initialize_Vnode)

设计思路:

每次对一个新序列进行验证时,必须清空上一轮节点的状态。将所有顶点的最短距离设为无穷大,状态设为未知,最后将给定的源点距离设为 0。

伪代码:

```
函数 Initialize_Vnode(Vnode, S)
输入: 顶点状态数组 Vnode, 序列起点 S
输出: 无

对于 i = 1 到 Graph.Nv:
    Vnode[i].dist = INFINITY
    Vnode[i].know = 0

Vnode[S].dist = 0
```

原代码:

```

void Initialize_Vnode(struct node *Vnode, int S) {
    for (int i = 1; i <= Graph.Nv; i++) {
        Vnode[i].dist = INFINITY; /*Reset distance to infinity*/
        Vnode[i].know = 0;        /*Reset status to unknown*/
    }
    Vnode[S].dist = 0; /*Distance of the starting point is 0*/
}

```

2.2.2 最小堆初始化 (Initialize_MinPQ)

设计思路：

由于除源点外的所有点初始距离都是无穷大，因此建堆时不需要进行耗时的排序操作（ $O(N)$ 建堆法）。直接将源点放置在堆顶（下标 1），其余节点按顺序放在后续位置，并同步建立 pos 数组的映射关系，即可保证符合最小堆的性质。

伪代码：

函数 Initialize_MinPQ(S)

输入：序列起点 S

输出：无

MinPQ.size = 1

对于 i = 1 到 Graph.Nv:

 如果 i != S:

 MinPQ.size = MinPQ.size + 1

 MinPQ.shortestPath[MinPQ.size].dist = INFINITY

 MinPQ.shortestPath[MinPQ.size].v = i

 MinPQ.pos[i] = MinPQ.size

 否则:

 MinPQ.shortestPath[1].dist = 0

 MinPQ.shortestPath[1].v = S

 MinPQ.pos[S] = 1

原代码：

```

void Initialize_MinPQ(int S) {
    MinPQ.size = 1; /*Start filling the MinPQ from index 1*/
    for (int i = 1; i <= Graph.Nv; i++) {
        if (i != S) { /*if i isn't the starting point ,insert into the tail of MinPQ*/
            MinPQ.size++;
            MinPQ.shortestPath[MinPQ.size].dist = INFINITY; /*initialization*/
            MinPQ.shortestPath[MinPQ.size].v = i; /*Recod the vertex of this distance*/
            MinPQ.pos[i] = MinPQ.size;          /*Record the position of vertex i*/
        }
        else {
            MinPQ.shortestPath[1].dist = 0; /*The distance of starting point is 0 */
            MinPQ.shortestPath[1].v = S;    /*Recod the vertex of this distance*/
            MinPQ.pos[S] = 1;               /*Record the position of vertex S*/
        }
    }
}

```

2.2.3 堆操作：交换（Swap）

设计思路：由于需要维护 `pos` 映射位置数组，在堆操作交换两个节点同时，需要更新**pos位置**，确保可以通过 `pos` 找到对应节点在最小堆里面对应的位置

伪代码：

```

函数 Swap(index1, index2)
输入：堆中的两个目标下标 index1 和 index2
输出：无

缓存 cache = MinPQ.shortestPath[index1]
MinPQ.shortestPath[index1] = MinPQ.shortestPath[index2]
MinPQ.shortestPath[index2] = cache

// 核心：同步更新 GPS 映射表
MinPQ.pos[MinPQ.shortestPath[index1].v] = index1
MinPQ.pos[MinPQ.shortestPath[index2].v] = index2

```

原代码：

```

void Swap(int index1, int index2) {
    // Swap the position of two node in minPQ
    struct MinPQNode cache = MinPQ.shortestPath[index1];
    MinPQ.shortestPath[index1] = MinPQ.shortestPath[index2];
    MinPQ.shortestPath[index2] = cache;

    void Sink(int index) {
        int left = index * 2;          /*Initialize the left child*/
        int right = index * 2 + 1; /*initialize the right child*/
        int small = index;             /*Initialize the smallest one of index left and right*/

        if (left <= MinPQ.size && MinPQ.shortestPath[small].dist >
            MinPQ.shortestPath[left].dist) {
            /*Update small index if the left child is in minPQ and is smaller*/
            small = left;
        }
        if (right <= MinPQ.size && MinPQ.shortestPath[small].dist >
            MinPQ.shortestPath[right].dist) {
            /*Update small index if the right child is in minPQ and is smaller*/
            small = right;
        }
        if (small != index) {
            /*Swap with the smallest child and continue to sink*/
            Swap(index, small);
            Sink(small);
        }
    }
}

// Update the position mapping array after swapping
MinPQ.pos[MinPQ.shortestPath[index1].v] = index1;
MinPQ.pos[MinPQ.shortestPath[index2].v] = index2;
}

```

2.2.4 堆操作：上浮与下沉 (Swim & Sink)

设计思路：

为了在节点移动时维护 `pos` 数组的准确性，所有节点交换通过封装的 `Swap` 函数，同时交换 `shortestPath` 中的结构体元素，同时更新 `pos` 数组的映射值。

Swim：当节点被松弛距离减小后，与其父节点（`index / 2`）比较，若更小则向上冒泡。

Sink：在 `Pop` 操作将末尾节点移至堆顶后，让其与其左右孩子（`index * 2` 和

`index * 2 + 1`) 比较, 选出三者中最小的进行交换, 向下沉淀。

伪代码:

函数 `Swim(index)`

输入: 当前需要上浮的节点在堆中的下标 `index`

输出: 无

如果 `index == 1`:

 返回 // 已到达堆顶, 上浮结束

// 如果当前节点比父节点 (`index / 2`) 的距离还要小

如果 `MinPQ.shortestPath[index].dist < MinPQ.shortestPath[index / 2].dist`:

 调用 `Swap(index, index / 2)` // 交换包裹并更新 `pos` 映射

 调用 `Swim(index / 2)` // 递归向上继续比较

函数 `Sink(index)`

输入: 当前需要下沉的节点在堆中的下标 `index`

输出: 无

`left = index * 2`

`right = index * 2 + 1`

`small = index`

// 检查左孩子是否存在, 且左孩子距离是否更小

如果 `left <= MinPQ.size` 且 `MinPQ.shortestPath[small].dist`
 `> MinPQ.shortestPath[left].dist`:

`small = left`

// 检查右孩子是否存在, 且右孩子距离是否比当前最小的还要小

如果 `right <= MinPQ.size` 且 `MinPQ.shortestPath[small].dist`
 `> MinPQ.shortestPath[right].dist`:

`small = right`

// 如果发现更小的孩子, 说明自身太大, 需要下沉

如果 `small != index`:

 调用 `Swap(index, small)` // 交换包裹并更新 `pos` 映射

 调用 `Sink(small)` // 递归向下继续沉淀

原代码:

```

void Swim(int index) {
    if (index == 1) {
        return; /*Stop when reach the top of the heap*/
    }
    if (MinPQ.shortestPath[index].dist < MinPQ.shortestPath[index / 2].dist) {
        /*Smaller than parent, swap and continue to swim*/
        Swap(index, index / 2);
        Swim(index / 2);
    }
}

void Sink(int index) {
    int left = index * 2;      /*Initialize the left child*/
    int right = index * 2 + 1; /*initialize the right child*/
    int small = index;        /*Initialize the smallest one*/

    if (left <= MinPQ.size && MinPQ.shortestPath[small].dist >
        MinPQ.shortestPath[left].dist) {
        /*Update small index if the left child is in minPQ and is smaller*/
        small = left;
    }
    if (right <= MinPQ.size && MinPQ.shortestPath[small].dist >
        MinPQ.shortestPath[right].dist) {
        /*Update small index if the right child is in minPQ and is smaller*/
        small = right;
    }
    if (small != index) {
        /*Swap with the smallest child and continue to sink*/
        Swap(index, small);
        Sink(small);
    }
}

```

2.2.5 稠密图验证 (Verification_Dense)

设计思路：

采用朴素的数组遍历寻找最小值。在每一次循环中，取出给定的序列元素 `currentV`，遍历整个 `Vnode` 数组。如果发现任何一个未确定顶点（`know == 0`）的距离严格小于 `currentV` 的

距离，说明 `currentV` 没资格在 `Dijkstra` 该位置，序列非法。验证通过后，将该点设为已确定，并遍历其邻接表进行松弛操作，更新其邻居的最短距离。

伪代码：

```
函数 Verification_Dense(Vnode, sequence)
输入：顶点状态数组，待验证序列
输出：序列合法返回 true，否则返回 false

对于 i = 0 到 Graph.Nv - 1:
    currentV = sequence[i]
    对于 j = 1 到 Graph.Nv:
        如果 Vnode[j].know == 0 且 Vnode[j].dist < Vnode[currentV].dist:
            返回 false

Vnode[currentV].know = 1
currentE = Graph.G[currentV].FirstEdge
当 currentE 存在时:
    如果 Vnode[currentE->AdjV].know == 0:
        如果 经过 currentV 到达邻居的距离更短:
            更新邻居的 dist
    currentE = currentE->next
返回 true
```

原代码：

```

bool Verification_Dense(struct node *Vnode, int *sequence) {
    bool vaildSequence = true; /*Initialize the vaild-singal*/
    for (int i = 0; i < Graph.Nv; i++) {
        int currentV = sequence[i]; // verify the sequence[i]

        for (int j = 1; j <= Graph.Nv; j++) {
            if (Vnode[j].know == 0 && Vnode[j].dist < Vnode[currentV].dist) {
                /*If we can found a closer unknown vertex, the sequence is invalid*/
                vaildSequence = false; /*update the singal*/
                return vaildSequence; /*Return to reduce time consumption*/
            }
        }

        Vnode[currentV].know = 1;
        PtrToAdjVNode currentE = Graph.G[currentV].FirstEdge;

        while (currentE) { /*Loop until all the ccontent is updated*/
            if (Vnode[currentE->AdjV].know == 0) { /*Only update the unknown vertex*/
                if (Vnode[currentE->AdjV].dist > Vnode[currentV].dist + currentE->weigh) {
                    /*Found a shorter parh and Update*/
                    Vnode[currentE->AdjV].dist = Vnode[currentV].dist + currentE->weigh;
                }
            }
            currentE = currentE->next; /*move the pointer to update next neighbor*/
        }
    }
    return vaildSequence;
}

```

2.2.6 稀疏图验证 (Verification_Sparse)

设计思路：

利用最小堆降低寻找最小值的复杂度。为了解决“合法性验证”，每次需要验证 `currentV` 时，先循环弹出堆顶那些已经在之前步骤中确定过（`know == 1`）的无用节点。当堆顶露出未确定节点时，其 `dist` 即为全局最低距离。对比 `currentV` 的距离是否等于堆顶距离，如果等于，说明该顶点可以被放在序列该位置，**即该点合法**，反之说明该点非法。验证成功后，进行松弛操作，更新邻居最短距离。最后通过通过 `pos` 数组定位邻居，修改其在最小堆中的数据，并调用 `Swim` 函数更新堆结构。

伪代码：

函数 `Verification_Sparse(Vnode, sequence)`

输入：顶点状态数组，待验证序列

输出：序列合法返回 `true`，否则返回 `false`

对于 `i = 0` 到 `Graph.Nv - 1`:

`currentV = sequence[i]`

 当 堆顶元素 `Vnode[MinPQ.shortestPath[1].v].know == 1` 时:

`Pop()` // 清理已确定的垃圾节点

 如果 `Vnode[currentV].dist != MinPQ.shortestPath[1].dist`:

 返回 `false`

`Vnode[currentV].know = 1`

`currentE = Graph.G[currentV].FirstEdge`

 当 `currentE` 存在时:

 如果邻居未确定 且 可以通过 `currentV` 获得更短路径:

 更新 `Vnode` 中邻居的 `dist`

 利用 `pos` 数组找到该邻居在堆中的位置 `posRefresh`

 更新堆中包裹的 `dist`

 调用 `Swim(posRefresh)` 让该节点上浮

`currentE = currentE->next`

返回 `true`

```

void Initialize_MinPQ(int S) {
    MinPQ.size = 1; /*Start filling the MinPQ from index 1*/
    for (int i = 1; i <= Graph.Nv; i++) {
        if (i != S) { /*if i isn't the starting point ,insert into the tail of MinPQ*/
            MinPQ.size++;
            MinPQ.shortestPath[MinPQ.size].dist = INFINITY; /*initialization*/
            MinPQ.shortestPath[MinPQ.size].v = i; /*Recod the vertex of this distance*/
            MinPQ.pos[i] = MinPQ.size; /*Record the position of vertex i*/
        }
        else {
            MinPQ.shortestPath[1].dist = 0; /*The distance of starting point is 0 */
            MinPQ.shortestPath[1].v = S;      /*Recod the vertex of this distance*/
            MinPQ.pos[S] = 1;                  /*Record the position of vertex S*/
        }
    }
}

```

2.2.7 测试主程序

设计思路：

- 初始化图结构和各个状态表，当使用最小堆验证时，再初始化最小堆
- 根据输入的图的稀疏程度决定使用哪种验证方式，当边数小于10倍的顶点数时，判定为稀疏图，采用最小堆验证法，当边数大于等于10倍顶点时，判定为稠密图，使用传统验证方法，最后输出结果。

```

int main() {
    int Nv, Ne;
    scanf("%d %d", &Nv, &Ne); /*Read the size of vertices and edges*/
    Graph.Nv = Nv;
    Graph.Ne = Ne;
    int v, w, distance;

    // Initialize the adjacency list for all vertices
    for (int i = 1; i <= Graph.Nv; i++) {
        Graph.G[i].FirstEdge = NULL;
    }

    // Build the undirected graph from the input
    for (int i = 0; i < Graph.Ne; i++) {
        scanf("%d %d %d", &v, &w, &distance);

        // build a ne node to represent the edge of v -> w
        PtrToAdjVNode neVW = (PtrToAdjVNode)malloc(sizeof(struct AdjVNode));
        neVW->AdjV = w;
        neVW->weigh = distance;
        neVW->next = Graph.G[v].FirstEdge;
        Graph.G[v].FirstEdge = neVW;

        // build a ne node to represent the edge of w -> v
        PtrToAdjVNode neWV = (PtrToAdjVNode)malloc(sizeof(struct AdjVNode));
        neWV->AdjV = v;
        neWV->weigh = distance;
        neWV->next = Graph.G[w].FirstEdge;
        Graph.G[w].FirstEdge = neWV;
    }

    int K, sequence[Nv];
    scanf("%d", &K); /*Read the number of sequences*/

    // Verify each sequence query
    for (int i = 0; i < K; i++) {
        for (int j = 0; j < Graph.Nv; j++) {
            scanf("%d", &sequence[j]);
        } /*Read and initialize the sequence*/
    }
}

```

```

Initialize_Vnode(Vnode, sequence[0]);
/*Initialize vertices status for the new sequence*/

bool vaild; /*The singal representing whether a sequence is vaild*/
if (Graph.Ne > 10 * Graph.Nv) {
    /*Use array-based verification for dense graph*/
    vaild = Verification_Dense(Vnode, sequence);
}
else {
    /*Use MinPQ verification for sparse graph to reduce time complexity*/
    Initialize_MinPQ(sequence[0]); /*Initialized the MinPQ*/
    vaild = Verification_Sparse(Vnode, sequence);
}

if (vaild) {
    printf("YES\n"); /*Valid Dijkstra sequence*/
}
else {
    printf("NO\n"); /*Invalid sequence*/
}
}
return 0;
}

```

Chapter 3：测试结果

3.1、测试样例与具体分析

测试样例文件	测试目的
test1.txt	综合测试，和题目给出的标准样例，确保基本图的构建与路径正确
test2.txt	验证同分情况（多分支路径长度一致），确保算法不会错误判断
test3.txt	验证非法越点：在最短距离未处理完时，指定较远节点

测试样例文件	测试目的
test4.txt	稀疏图架构验证，验证该方法可以正确判断序列合法性
test5.txt	稠密图架构验证，验证该方法可以正确判断序列合法性
test6.txt	边界情况测试：整个图为一个孤立点
test7（8）.txt	边界情况测试：数据规模接近题目上限验证是否可以正常运行

3.1.1 测试样例 1：综合测试 (test1.txt)

```
5 7
1 2 2
1 5 1
2 3 1
2 4 1
2 5 2
3 5 1
3 4 1
4
5 1 3 4 2
5 3 1 2 4
2 3 4 5 1
3 2 1 5 4
```

预期结果：

```
YES
YES
YES
NO
```

测试样例说明：标准无向图测试，验证基本的数组更新和起点判定（由序列首元素触发）。

3.1.2 测试样例 2：同分情况验证 (test2.txt)

```
4 4
1 2 2
1 3 2
2 4 1
3 4 1
2
1 2 3 4
1 3 2 4
```

预期结果：

```
YES
YES
```

测试样例说明：点 2 和点 3 距离源点 1 的距离同为 2。验证无论序列是先选 2 还是先选 3，验证系统都能识别两者均为合法。

3.1.3 测试样例 3：非法越点验证 (test3.txt)

```
4 3
1 2 1
2 3 1
3 4 1
1
1 3 2 4
```

预期结果：

```
NO
```

测试样例说明：到达点 3 的距离必须先经过点 2。在点 2 还未确认 (know=0) 时选取点 3，程序应在与堆顶数据比对中精确判断该点不合法并返回 false。

3.1.4 测试样例 4：稀疏图架构验证 (test4.txt)

测试目的：针对题目边数条件 $E \leq 10V$ 构建的稀疏图测试。验证程序是否能正确判断图的密度，切入 `Verification_Sparse` 最小堆优化分支，并依靠 `pos` 数组映射和懒惰删除法正确验证序列。

输入数据：

```
5 4
1 2 1
2 3 1
3 4 1
4 5 1
2
1 2 3 4 5
1 3 2 4 5
```

预期结果：

```
YES
NO
```

测试样例说明：验证的链状稀疏图 ($V = 5, E = 4$)，测试稀疏图验证架构有无问题。序列 1 正确按距离递增顺序给出Dijkstra序列；序列 2 试图在点 2 未确定前越级确认点 3，触发堆顶最小值校验失败，并返回 `false`。

3.1.5 测试样例 5：稠密图架构验证 (test5.txt)

测试目的：针对题目边数条件 $E > 10V$ 构建的稠密图测试。为了触发该分支，顶点数 $V \geq 22$ 且边数逼近完全图。验证程序是否能正确切入 `Verification_Dense` 验证分支，并正确输出结果。

预期输出：由测试程序控制，奇数行为正确序列，偶数行为错误序列。

测试样例说明：由于该样例过长，不在报告里呈现，详见测试文件test5.txt

注：该样例由脚本生成，用于测试不同数据规模情况下的代码运行情况，仅作生成大规模数据使用，仅供参考

3.1.6 测试样例 6:边界情况验证 (test6.txt)

测试目的：检测整个图为一个单独的点时，程序是否可以正常运行

输入数据：

```
1 0
1
1
```

预期结果：

```
YES
```

3.1.7 测试样例 7：边界情况验证 (test7.txt & test8.txt)

测试目的：检测整个图在数据规模接近题目上限时，程序是否可以正常运行，稠密图和稀疏图验证架构正常运行

预期输出：由测试程序控制，奇数行为正确序列，偶数行为错误序列。

测试样例说明：由于该组样例过长，不在报告里呈现，详见测试文件test5.txt

注：该组样例由脚本生成，用于测试不同数据规模情况下的代码运行情况，仅作生成大规模数据使用，仅供参考

Chapter 4：复杂度分析

4.1、各函数复杂度分析

4.1.1、构建图与初始化 (Main/Initialize)

1. 时间复杂度 $O(|V| + |E|)$ ：循环遍历顶点初始化指针，随后遍历所有边使用头插法插入邻

接表，由于是无向图，**每条边处理两次**，总时间为 $O(|V| + |E|)$ 。

2. **空间复杂度 $O(|V| + |E|)$** ：需要使用数组存储所有顶点状态，并为每条边动态分配链表节点。

4.1.2、堆操作——交换 (swap)

1. **时间复杂度 $O(1)$** ：无递归，无循环
2. **空间复杂度 $O(1)$** ：仅需要单独的MinPQNode节点进行中转

4.1.3、堆操作——上浮 (swim)

1. **时间复杂度 $O(\log |V|)$** ：最小堆属于完全二叉树结构，树高度为 $\log_2 |V|$ ，最坏情况，通过递归，将一个叶节点向上浮到堆顶，共需要对比 $\log_2 |V|$ 次，也就是时间复杂度为 $O(\log |V|)$
2. **空间复杂度 $O(\log |V|)$** ：最坏情况递归深度为树的高度即 $\log_2 |V|$ ，总空间复杂度为 $O(\log |V|)$

4.1.4、堆操作——下沉 (sink)

1. **时间复杂度 $O(\log |V|)$** ：与上浮操作类似，最坏情况，将堆顶元素下沉，直到达到叶节点，下落层数为 $\log_2 |V|$ ，每一层与两个子节点比较，最多比较次数为 $2\log_2 |V|$ ，忽略常数最终时间复杂度为 $O(\log |V|)$
2. **空间复杂度 $O(\log |V|)$** ：最坏情况递归深度为树的高度即 $\log_2 |V|$ ，总空间复杂度为 $O(\log |V|)$

4.1.5、原始数组验证 (Verification_Dense)

1. **时间复杂度 $O(|V|^2 + |E|)$** ：外层按序列遍历 V 次；内层循环寻找最小距离遍历全图，进行 V 次比较；松弛操作更新邻居距离整体遍历所有的边 E 次。总时间为 $O(|V|^2 + |E|)$ 。
2. **空间复杂度 $O(1)$** ：没有使用额外数据结构空间消耗，仅利用已有邻接表和状态表。

4.1.6、最小堆验证 (Verification_Sparse)

1. **时间复杂度 $O(|E| \log |V|)$** ：1.验证耗时：外层遍历 V 次验证顶点；清理堆 (sink) 的时间为 $O(\log |V|)$ ；2.更新堆耗时：每条边最多触发一次更新 (swim)，最坏发生 E 次，每次 $O(\log |V|)$ ，最后时间复杂为 $O(|E| \log |V|)$ ，综上所述，最终时间复杂度为 $O(|E| \log |V|)$
2. **空间复杂度 $O(V)$** ：需要额外开辟大小为 $|V|$ 的堆数组 `shortestPath` 以及位置映射数组

pos。

4.2、总结

函数/方法	时间复杂度	空间复杂度	说明
Verification_Dense	$O(V^2 + E)$	$O(1)$ 额外空间	朴素数组验证，利用两层 <code>for</code> 循环寻找最小值
Verification_Sparse	$O(E \log V)$	$O(V)$	利用带映射的最小堆，支持直接更新距离改变量， $O(\log V)$ 上浮，下沉操作。
Swim / Sink	$O(\log V)$	$O(\log V)$	维护最小堆性质，同时在交换时同步更新 <code>pos</code>

4.3、总体复杂度

程序具备动态验证的能力，能够根据图的稀疏程度自动切换验证方法。

- **时间复杂度**：当 $E > 10V$ 时使用密集验证，整体时间 $O(V^2 + E)$ ；当 $E \leq 10V$ 时使用堆优化，整体时间 $O(E \log V)$ 。
- **空间复杂度**：最坏情况下均需存储邻接表结构，总体空间复杂度为 $O(V + E)$ 。

图稀疏程度选取 $|E| = 10|V|$ 为分界线 的原因：

- 在纯大 O 渐进复杂度理论中，数组版时间复杂度为 $V^2 + E$ ，堆优化版时间复杂度为 $E \log_2 V$ 。令两者相等可推导出数学临界点为 $E \approx V^2 / \log_2 V$ 。
- 但根据查阅资料我了解到由于硬件限制理论与实际速度会有区别，数组验证法，内存访问是线性的，进行简单的读写和比较，速度很快，而最小堆算法，父子节点的跳转内存是不连续的，索引计算，缓存不命中会造成大规模的延迟所以，除非**该图极其稀疏**，使得图的结构类似于树，此时使用最小堆优化效率才有价值。
- 最后查询工业界常用的稀疏稠密分界值：基础设施与底层网络（ $E = 2 \sim 5V$ ），信息与数据网路（ $E = 5 \sim 50V$ ）等等，选取折中 $E = 10V$ 作为本题的分界线，保证了使用**最小堆优化**可以提高算法效率

4.4、反思与改进

- **最小堆结构的改变**：引入了位置映射数组 `pos`，增加了代码的复杂程度，但也提高了空间利用率，实际应用可结合需要调整使用
 - **堆操作使用尾递归**：本题使用递归算法进行堆的**上浮**和**下沉**操作，当数据量极大（超出本题范围）可能会有爆栈风险，因此可改为迭代算法，避免该问题
 - **硬件限制**：最小堆优化需要图的稀疏程度**理论值**和**实际值**存在较大差距，需要考虑硬件限制对算法时间复杂度的限制，以后优化算法需要考虑硬件限制对于常数项开销的影响
-

声明

我在此声明项目“*Dijkstra Sequence*”里的所有内容均由我自己独立完成。

I hereby declare that all the work done in this project titled "*Dijkstra Sequence*" is of my independent effort.